

Monitoring & Alert Pipeline

How JSAN Data Center OpsDesk detects, correlates, pages, and resolves infrastructure issues — and where each stage surfaces as a visualization.

*Reflects the alerts, monitoring, incidents, and reports modules as implemented (apps/api/src/modules/{alerts,monitoring,incidents,reports})
— not a design proposal.*

The loop

Every signal moves through the same four-stage loop the product's own Command Center diagrams: site collectors **observe** the estate without exposing it, ingestion **correlates** a raw event into a deduplicated, normalized alert, the service desk **responds** through a governed incident state machine, and the platform **learns** by closing the loop — confirming the underlying condition actually cleared, not just that a ticket was clicked closed.



In this document

1	End-to-end pipeline	Every stage from raw signal to a visualized, reported outcome.
2	Alert lifecycle & severity	The OPEN → ACKNOWLEDGED → RECOVERED state machine, and how correlation actually works.
3	Where it's visualized	The four surfaces reading this data, and the design choices behind their charts.
4	Integrity & governance	The non-negotiables that keep this pipeline trustworthy under load.

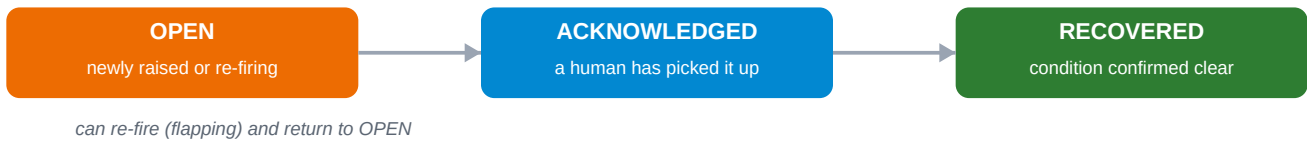
End-to-end pipeline



Each node is a real code path, not an aspirational step: source → AlertsService.ingest*() → AlertRule lookup → Alert row → IncidentsService.findOpenByCi() / linkAlert() → NotificationsPublisher → IncidentsService.createTransition() → IncidentsService.notifyAlertRecovered().

Alert lifecycle

An alert's own state is tracked independently of any incident it gets linked to. The same fingerprint carries an alert through re-firing (flapping) without spawning duplicates — only the state and lastSeenAt move.



Severity ladder

Severity is set by the source (or, for hardware health, derived from the Redfish/iDRAC reading) and never changes on recovery — a resolved CRITICAL stays CRITICAL, just RECOVERED, because severity records what the condition was.

Severity	Meaning	Paged to NOC by default?
■ CRITICAL	Service-impacting now	Yes
■ HIGH	Degraded, at risk of impact	Per alert rule
■ WARNING	Worth tracking, not yet urgent	No
■ INFO	Informational signal	No

Correlation, precisely

Correlation is **link-only**: when a new alert lands on a CI that already has an open incident, the alert is attached to it (correlatedIncidentId) so the ticket shows every signal behind it. It never auto-creates a ticket — that stays a human decision in the Service Desk queue, keeping the incidents module the sole owner of ticket business rules (CLAUDE.md module boundaries: alerts owns normalized signals, incidents owns state).

Where it's visualized

Four surfaces read from this same pipeline, each scoped by the viewer's site access server-side — no surface trusts the frontend to hide what a role shouldn't see.

Surface	What it shows	Refresh
Alerts queue (/alerts)	Raw, filterable list of every normalized alert — state, severity, source, CI, linked incident.	On demand
Command Center (/)	Live site health rollup, open critical-alert count, SLA/queue counters; downloadable operational-health CSV.	On load
Alert Insights (/alert-insights)	Severity & state breakdown and a daily new-alert volume trend, both scoped to one 7/30/90-day window.	On demand, per window
Incident detail (/incidents/:id)	Linked-alerts panel on the ticket itself — what triggered it, and whether it has actually recovered.	Polled while open

Design notes on the charts themselves

Severity and state are shown as labeled, color-coded stat tiles rather than another pie chart — a handful of headline counts reads faster as tiles than as slices competing for the same four categories. The daily trend is a stacked bar (not overlaid lines), which keeps four simultaneous severities visually separated by the bar segments themselves, not color alone, and a data table sits under every chart so no number is reachable only by hovering.

Integrity & governance

- **Idempotent by construction.** Every ingestion path dedupes on (source, external_event_id) plus the fingerprint hash, so a retried webhook or a flapping condition never creates duplicate alerts.
- **Config over hard-code.** Flapping thresholds, which severities page the NOC, auto-correlation, and maintenance-window suppression all live in the **AlertRule** table — resolved most-specific match first (site+type → site → type → global), never a code constant.
- **Alert state is independent of ticket state.** Resolving the incident does not resolve the alert. The resolve gate blocks marking a ticket RESOLVED while its alert is still OPEN unless a human explicitly overrides with a reason — and if that alert later recovers on its own, the team is notified so the loop actually closes.
- **No destructive hardware actions.** Redfish/iDRAC/iLO integration is read/observe/record only — no firmware updates, reboots, or BIOS changes from the portal.
- **Management interfaces never face the Internet.** iDRAC/iLO/SNMP stay behind the site collector, which only connects outbound over TLS/mTLS.
- **Every state change is audited.** Alert correlation, NOC paging, and every incident transition write an append-only audit event — nothing is inferred after the fact.